PXE Boot on UniFi — Juniper Imaging Infrastructure

Troubleshooting, fix documentation, and deployment automation for native PXE boot on Juniper's UniFi network.

Current Status (2026-06-19)

Root cause confirmed and fix applied — pending live boot test.

EFG DHCP was serving `https://192.168.5.141/shimx64.efi` as the TFTP boot filename (Option 67). PXEBOOT firmware clients treat Option 67 as a TFTP filename and literally requested the URL string via TFTP. tftpd64 received an invalid Windows path (`https:...`), threw an exception, and silently dropped every request — zero log output, zero TFTP response.

Fix applied 2026-06-19 via UniFi UI: Option 67 changed to `shimx64.efi`. See PXE-FINDINGS.md for the full diagnostic record.

Stack

Component	Host	Details
EFG Router / DHCP	192.168.0.1	UniFi OS — serves Options 43/66/67 for PXEBOOT
pc-deploy	192.168.5.141	Windows 11, tftpd64 + Caddy HTTPS + deploy share
tftpd64	pc-deploy	Serves `C:\tftpd64\`, UDP 69, TFTP only (not proxy DHCP)
Caddy	pc-deploy	HTTPS on 443, HTTP on 80, serves boot.wim + deploy share
shimx64.efi	`C:\tftpd64\`	Ubuntu shim (MS dual-signed), first TFTP payload
grubx64.efi	`C:\tftpd64\`	iPXE, signed by Juniper CA, chainloaded by shim
boot.wim	`C:\tftpd64\sources\`	WinPE (NIC drivers injected, ~513 MB), fetched via HTTP
deploy share	`\\192.168.5.141\deploy$`	Windows images, scripts, unattend XMLs
Inventory API	`http://192.168.5.141:8080`	Device identity lookup / imaging log ingest

Boot Chain

```
Target UEFI → PXEBOOT
    → DHCP ACK: filename=shimx64.efi, siaddr=192.168.5.141
    → TFTP RRQ shimx64.efi → served from C:\tftpd64\
    → shim loads grubx64.efi (Juniper-signed iPXE)  [MOK required]
    → iPXE runs autoexec.ipxe → HTTP fetch wimboot + boot.wim
    → WinPE boots
    → startnet.cmd: maps \\192.168.5.141\deploy$ → runs deploy.ps1
    → deploy.ps1: inventory preflight → OS selection → DISM image apply
    → post-install scripts: 03-07*.ps1
    → JuniperInventoryAgent.msi installed → agent reports to inventory server
```

Deployment Automation

`scripts/deploy.ps1` is the core imaging script, run from WinPE. It has evolved significantly:

- **Inventory preflight** — queries `http://192.168.5.141:8080` by serial number before prompting
- **30-second countdown** — shows previous hostname/OS as defaults; operator can override
- **MAC-based dedup** (fixed 2026-06-17) — includes `ethernet_macs` + `wireless_macs` in the osPatch POST so the inventory server uses MAC as primary identity, preventing ghost records from serial-only lookups
- **UEFI detection** — logs whether the target booted in UEFI or legacy mode
- **osPatch endpoint** — POSTs imaging result back to inventory with machine identity

`scripts/orchestrator.ps1` and `scripts/setup-inventory-native.ps1` handle inventory server setup.

`winpe/deploy-boot.ps1` runs inside WinPE (baked into boot.wim) and bootstraps the share connection before handing off to `deploy.ps1`.

Secure Boot

All target machines at Juniper use UEFI Secure Boot. The shim chain requires per-machine MOK enrollment:

- **Yoga 7 2-in-1 (192.168.11.3):** MOK enrolled 2026-06-18 ✅
- **ThinkPad P14s Gen 5 (192.168.11.24):** MOK enrollment pending

Run `push-mok-enrollment.ps1` against a machine after it's online to enroll `juniper-pxe-ca.cer`.

Network

```
EFG Router (192.168.0.1)  /20 flat network — 192.168.0.0–192.168.15.255
    └── USW Pro 48 PoE
            ├── UniFi U7 Pro XG APs (× 5)
            └── Port 21 → TP-Link TL-SG108-M2 (Engineering — unmanaged)
                    ├── pc-deploy         192.168.5.141
                    ├── ThinkPad P14s     192.168.11.24
                    └── Yoga 7 2-in-1     192.168.11.3  (WiFi only)
```

No VLANs. No DHCP relay. All machines on the same /20 scope.

Repository History

This repo (`jasonjuniper/pc-imaging-server`) has a merged history from two sources:

1. **PXE boot investigation** (2026-06-18–19) — diagnostic scripts, tftpd64 config, shim chain setup, PXE-FINDINGS.md
2. **Deployment automation** (from `C:\dev\pc-imaging-server`, 2026-06-10–17) — deploy.ps1, orchestrator.ps1, WinPE scripts, driver manifests, inventory agent integration

The merge was necessary because both histories lived in the same GitHub remote (`jasonjuniper/pc-imaging-server`). See commit `e64ebcf`.

Key Files

File	Purpose
`PXE-FINDINGS.md`	Full diagnostic record — all hypotheses, test results, root cause, fix
`scripts/deploy.ps1`	WinPE imaging script (inventory preflight, OS selection, DISM apply)
`scripts/orchestrator.ps1`	Inventory server orchestration
`winpe/deploy-boot.ps1`	WinPE bootstrap (baked into boot.wim)
`winpe/startnet.cmd`	WinPE startnet — maps share, launches deploy-boot.ps1
`scripts/01c-build-winpe.ps1`	Builds boot.wim, injects NIC drivers
`push-mok-enrollment.ps1`	Enrolls Juniper CA cert for Secure Boot MOK on a target machine
`scripts/wim-bake-credentials.ps1`	Bakes junadmin credentials into boot.wim

Juniper Design — internal infrastructure